

Efficient Instruction Vulnerability Prediction With Heterogeneous SDC Propagation Knowledge Graph

Bao Wen[✉], Jingjing Gu[✉], *Member, IEEE*, Dazhong Shen[✉], *Member, IEEE*, Qiang Zhou[✉],
Fuzhen Zhuang[✉], *Member, IEEE*, Yang Liu[✉], Haocheng Song, and Xinyi Huang[✉], *Member, IEEE*

Abstract—With System on Chip integration on the rise, Silent Data Corruption (SDC) poses a significant threat to computer systems, corrupting outputs silently and without clear faults. Traditional error detection methods lack either energy efficiency or accuracy, failing to capture SDC's complex propagation patterns. To the end, in this paper, we propose a new paradigm called VP-HPKG, which leverages a Heterogeneous Program Knowledge Graph to intricately map structural interdependencies between basic blocks and instructions, enabling the exploration of potential SDC propagation paths. Specifically, first, we build an instruction execution and register fault generation system, based on which we can simulate bit-flip errors to obtain data such as program semantic information and execution status. Second, due to complex inter-instruction relations and random error propagation, we construct a multi-layer heterogeneous program knowledge graph by characterizing entities and relations of instructions, which implicates the potential path of error propagation. Then, we model contextual correlations within and among basic blocks with Graph Neural Network and Transformer, aiming to uncover abnormal inter-block jumps and extract the instruction embedding to predict vulnerable instructions within blocks accurately with low overhead. In particular, we perform 1,144,070 fault injections based on LLFI to obtain sufficient samples of SDCs. Our experimental results with 22 programs show our method outperforms the state-of-the-art baselines in terms of accuracy (average 10.3%↑), F1-score (average 18.4%↑). In addition, the model complexity of VP-HPKG is reduced by about 2 times compared to the most competitive method, and the fault injection overhead is reduced by 30%.

Index Terms—Instruction vulnerability prediction, silent data corruption, error propagation, knowledge graphs.

Received 16 April 2024; revised 10 September 2025; accepted 22 September 2025. Date of publication 25 September 2025; date of current version 14 January 2026. This work was supported in part by the China Postdoctoral Science Foundation under Grant GZC20252740, in part by the Jiangsu Funding Program for Excellent Postdoctoral Talent, and in part by the Fundamental Research Funds for the Central Universities under Grant NJ2024031. (*Corresponding author: Jingjing Gu.*)

Bao Wen, Dazhong Shen, Yang Liu, Haocheng Song, and Xinyi Huang are with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China (e-mail: wenbao@nuaa.edu.cn; shendazhong@nuaa.edu.cn; liuyaaa@nuaa.edu.cn; songhc-nuaaei@nuaa.edu.cn; xyhuang81@gmail.com).

Jingjing Gu and Qiang Zhou are with the College of Computer Science and Technology and MIT Key Laboratory of Pattern Analysis and Machine Intelligence, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China (e-mail: gujingjing@nuaa.edu.cn; zhouqnuaacs@nuaa.edu.cn).

Fuzhen Zhuang is with the Institute of Artificial Intelligence, Beihang University, Beijing, China, and also with Zhongguancun Laboratory, Beijing, China (e-mail: zhuangfuzhen@buaa.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3614343

I. INTRODUCTION

WITH the exponential increase of the System on Chip (SoC) integration, the stability of the system is increasingly challenged [1], [2], [3]. The electromagnetic radiation, hardware faults, and software errors all contribute to transient faults, which can generate a variety of system errors. It has been demonstrated that transient faults constitute the majority of errors in modern computing systems (up to 97.84%) [4]. Among these errors, the Silent Data Corruption (SDC) is one of the very frequent and serious issues [5], [6], [7], where the corrupted program finishes execution but produces erroneous results silently. As it is difficult to be detected by the system, it has received a lot of attention from researchers at Meta,¹ Google, etc.² in the past two years. Specifically, hardware faults (e.g., bit flips) can be propagated to various components through the operation of instructions, generating SDC errors. Meanwhile, software errors (e.g., data errors and security attacks) can also lead to the generation and propagation of SDC errors if they are not detected promptly and effectively. In particular, these issues have become more serious recently due to the growing complexity of the various systems [8], where the number of SDC errors has increased dramatically.

Researchers have observed that a portion of program instructions is responsible for almost all SDC errors in a program [9]. Therefore, protecting the most vulnerable portion of instructions in a program may be sufficient to achieve a high SDC detection rate with relatively low overhead. Given this, instruction vulnerability prediction (which aims to estimate the probability that an instruction may result in SDC in the event of an error) has gained wide attention in the last few decades. The most common traditional approach is based on hardware fault injection [10], [11], which generates SDC errors by simulating hardware faults and identifies vulnerable instructions through statistical analysis. However, it requires full fault injection of program instructions, flipping every bit of each instruction, so its energy consumption grows exponentially with the increase in program size and dynamic instructions. Thus, researchers have proposed several methods based on machine learning and neural networks recently [12], [13], [14], which build a dataset by performing partial fault injection on program instructions to

¹<https://research.facebook.com/research-awards/2022-silent-data-corruptions-at-scale-request-for-proposals/>

²<https://www.iscaconf.org/isca2022/program/>

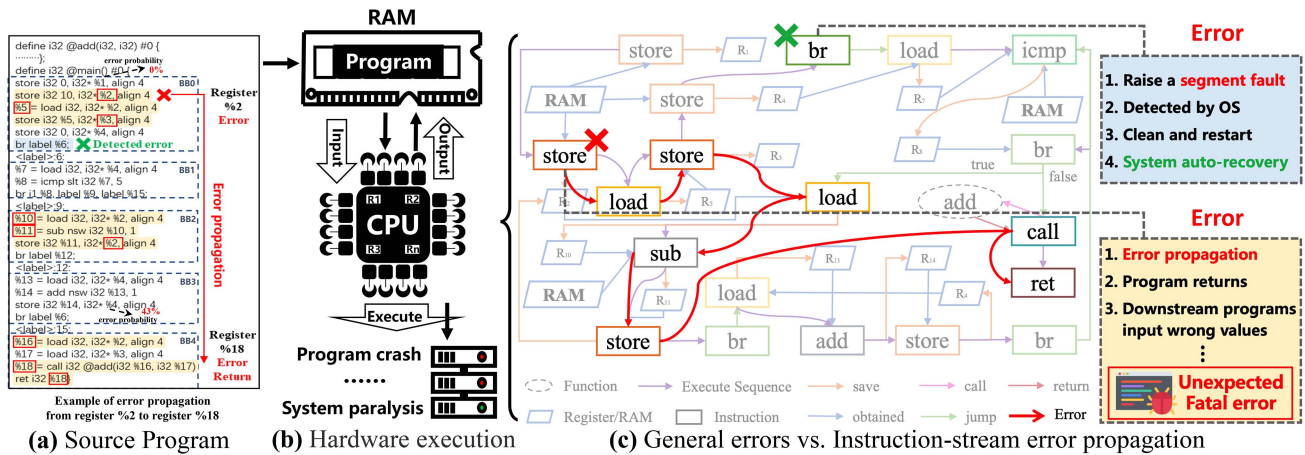


Fig. 1. Schematic diagram of the SDC error generation and propagation.

train the model and identify error-prone instructions. Unfortunately, while the reduction of energy consumption is a notable achievement, there are still several challenges:

Challenge 1: Complex error propagation patterns. Indeed, due to the dynamic nature of instruction execution, the propagation of errors can be extremely complex, making it difficult to predict and locate the vulnerable instructions. Specifically, when an error is generated, it may lead to a program crash immediately or propagate backward randomly as program execution, which makes it challenging to predict the instruction vulnerability accurately. For instance, as shown in Fig. 1(a), (c), when the register %6 on the instruction “*br, label%6*” makes an error, it is directly detected by the system. However, when the register %2 on the instruction “*store10, i32 * %2, align4*” makes an error, it will not be detected by the system but propagated to registers operated by the subsequent instructions as program execution, such as the propagation path of $\%2 \rightarrow \%5 \cdots \rightarrow \%18$.

Challenge 2: Position-sensitive instruction vulnerability. On the other hand, different components, transmission channels, and basic blocks would be involved in different probabilities of errors, even for the same instruction in different positions of *control flow* or *data flow*. For example, the “*store*” instruction in Fig. 1(a) in different program basic blocks has a different probability of errors during the execution of the program. As a result, it is also a challenging problem to locate the basic block containing the vulnerable instruction and track the whole propagation path, which can provide better interpretability for error analysis.

To address the above challenges, we provide a new paradigm for Efficient Instruction Vulnerability Prediction with Heterogeneous SDC Propagation Knowledge Graph (VP-HPKG), from the perspective of registers, since it is laid in the critical path of instruction stream as in Fig. 1(b). Specifically, first, based on LLVM [15] and LLFI [16], we build an instruction execution and register fault generation system, where we can simulate bit-flip errors to obtain data such as program information and execution status. Then, we exploit structural and non-structural program information and construct a multi-layer heterogeneous program knowledge graph, which implicates the potential path

of error propagation. Instructions and basic blocks are entities and multiple relations in terms of *control flow*, *data flow*, *jump*, *call*, and *access* are involved. In particular, instructions in different basic blocks are distinguished, which can provide more refined information for locating and tracking the instructions that will result in SDC. Finally, based on this knowledge graph, we enhance the instruction embedding by capturing intra- and inter-block contextual correlations with a combination of Graph Neural Networks (GNNs) and Transformers. This facilitates the detection of abnormal inter-block jumps and improves the accuracy of instruction vulnerability prediction within blocks.

The experimental results on the 22 programs show the effectiveness and efficiency of VP-HPKG compared to the state-of-the-art methods. Subsequently, the location and propagation path of instructions can be determined based on our knowledge graph easily. Our contributions are summarized as follows:

- We propose VP-HPKG, a novel paradigm for efficient instruction vulnerability prediction by leveraging heterogeneous SDC propagation knowledge graphs and graph neural networks.
- To our knowledge, we are the first to implement fine-grained SDC error propagation modeling by using a multi-layer heterogeneous program knowledge graph. It integrates various relations such as control flow, data flow, calls, jumps, and accesses, while distinguishing the instructions of each basic block.
- We develop a hybrid graph-based encoder combining GNN and Transformer to jointly model intra- and inter-block contextual dependencies, which improves both the accuracy and interpretability of vulnerability prediction.
- We build an instruction execution and fault generation system, which performs bit flips during the runtime of programs. Extensive experiments with 22 programs are conducted to validate the effectiveness of our method. Compared to the most competitive baseline, our method improves 10.3% in *Acc* and 18.4% in *F1*. The verifiable data and code are published in <https://github.com/SINCOSLab/VP-HPKG>.

II. RELATED WORK

This section summarizes the existing literature related to this work, which includes traditional vulnerability prediction based on fault injection and artificial intelligence-based vulnerability prediction.

A. Vulnerability Prediction Based on Fault Injection

The method based on fault injection generated errors by simulating hardware faults and identified vulnerable instructions through statistical analysis [17], [18], [19]. For example, Santos et al. [20] created a pool of possible fault effects on the operation output based on the instruction opcode and input characteristics, and then injected these fault effects into programs using a software framework. They reduced the fault injection time, thus allowing, for the first time on GPUs, to track the fault propagation from the hardware to the output of complex applications. Yuan et al. [17] accurately and efficiently simulated faults on processors by dynamically switching, with little designer effort, enabling fast identification of the portions that would benefit most from hardening. Hukerikar et al. [19] introduced a novel approach to optimizing large-scale fault injection experiments by leveraging the Martingale hypothesis. By integrating probabilistic models, they enhanced the efficiency of fault injection and enabled deriving high-confidence estimates of the diagnostic capabilities of systems. Joshi et al. [18] present a systematic framework that combines empirical fault injection and propagation analyses to enable fast error analysis of evolving programs. It can reduce the burden of repeated fault injection whenever developers add protections for vulnerable instructions. However, the cost of hardware fault injection increased with program size.

B. Vulnerability Prediction Based on Artificial Intelligence

In recent years, machine learning models have been widely employed to predict vulnerable instructions [12], [21], [22], thereby reducing the space-time overhead while ensuring accuracy. Yan et al. [22] used the one-class support vector machine and instruction propagation features to identify vulnerable instructions, aiming to protect the data in the memory or register used by the program. Jiang et al. [23] chunked the program into fixed-size segments and aggregated the embeddings of these segments, understanding the context information of long code sequences typical in high-performance computing systems. Tan et al. [21] employed the extracted heuristic property vector as inputs to train a classifier for final fault outcomes in advance.

Nevertheless, the selection of features in these methods depended on the researcher's own understanding, and there was no way to ensure the effectiveness of these features for the downstream task. Therefore, the method of instruction vulnerability prediction based on program structure information has received much attention. Jiao et al. [24] exploited the synergy between static analysis and data-driven statistical inference to automatically identify instruction-level vulnerabilities using fine-grained error propagation information from a set of bit-level program graphs. Wei et al. [25] constructed a dependency graph for the

general-purpose graphics processing units program, and drove a graph attention network to predict SDC proneness of instructions under the transient fault influences. Yu et al. [26] exploited the propagation of bit-level faults during instruction execution and data transfer processes, and used graph neural networks and instruction semantics to capture their distinct error propagation patterns. By leveraging the control data flow graph and calling context tree to represent the commonalities and distinctions between different input executions, Ma et al. [27] calculated SDC probabilities of instructions through a structured graph attention network. However, the numerous connections between instructions, the attributes of instructions themselves, or the intricate structural details of the program are not taken into account.

III. BACKGROUND

A. Fault Model

In this paper, we consider transient hardware faults occurring in the processor's computational components, including flip-flops and functional units, which interact with registers either directly or indirectly. Thus, we specifically focus on bit flips occurring in the registers manipulated by each instruction. Faults in memory or caches are excluded from our study, as these are assumed to be protected by Error Correcting Code (ECC). Similarly, we assume an ECC-protected register file, thereby constraining potential bit flips to the transient manipulation of data within the execution unit. Faults in processor control logic are considered out of scope due to the presence of internal redundancy or verification mechanisms. Additionally, we exclude faults in instruction encoding [28], [29], as they are often identified and mitigated through low-level protection techniques such as checksumming or fetch-time validation. Finally, although bit flips in micro-architectural registers can lead to distinctive error patterns (e.g., illegal jumps) [30], this study focuses on the impact of bit flips on data computation processes within programs, and thus excludes them. Our fault model is consistent with other work in the area [31], [9], [32], [33]. The fault model F^M can be expressed as a quadruple as follows:

$$F^M = \{S_e, F_T, N^T, Reg\}, \quad (1)$$

where S_e is the instruction set for fault injection, F_T is the type of fault (i.e., bit flips), N^T represents the total number of fault injections, and Reg is the registers for fault injection, including the destination and source registers.

B. Fault Effect

To assess the impact of injected faults, we adopt a commonly used taxonomy to classify fault outcomes [14], [22], [31]:

- Benign: The fault that does not cause an observable deviation from the expected system behaviour, meaning the program produces the correct output. (i.e., the error was either masked or handled by the program)
- Silent Data Corruption (SDC): The fault that causes the program to produce an incorrect output, but with no indication that the fault has occurred.

TABLE I
FREQUENTLY USED NOTATIONS

Notations	Descriptions
N	Total number of instructions.
M	Total number of basic blocks.
J	The number of relations;
A_p	The set of all attributes.
G	A knowledge graph with V and E .
V	The set of vertices in G .
E	The set of edges in G .
P_I	The error rate of instructions.
g_B	A subgraph of basic block $g_B \in G$.
g_I	A subgraph of instruction $g_I \in G$.
n_i	An instruction entity, $i = 1, 2, \dots, N$.
B_i	A basic block entity, $i = 1, 2, \dots, M$.
b_i	The attribute of B_i .
I_i	The attribute of n_i .
B'	The set of basic block embedding.
n''	The set of instruction embedding.
W_h	Weight of the h -th stratum.

- Crash: The fault that causes the program to terminate prematurely, either by triggering an exception or executing an unintended exit path (e.g., due to a faulty branch decision) before completing its intended execution.
- Hang: The fault that causes an infinite loop or abnormally prolonged execution that ultimately results in termination due to exceeding a predefined timeout threshold.

To better understand the underlying causes and system responses, it is useful to categorize errors based on their detectability and recoverability.

- Detected errors: Errors identified by hardware or software mechanisms, such as ECC and watchdog timers (e.g., Hang).
- Undetected errors: Errors that escape detection and may silently affect program output or behavior (e.g., SDC).
- Detected and corrected errors: Errors that are not only detected but also automatically corrected, allowing execution to proceed normally (e.g., Benign).
- Detected and uncorrectable errors: Errors that are detected but cannot be corrected, typically triggering exceptions or halting execution (e.g., Crash).

Among them, SDC is particularly important because it represents undetected errors that silently corrupt the output of a program without any alerts, posing a major threat to system reliability. Thus, this paper focuses primarily on predicting instructions that are prone to SDC.

C. Problem Definition

We list the main variable notations in Table I. Given a program S , which consists of basic blocks $BB = \{B_1, B_2, \dots, B_M\}$, where M is the total number of basic blocks. Each basic block B_j consists of multiple script lines, which can be compiled into a set of instructions $\{n_i | n_i \in B_j, n_i \in IN\}$, where $IN = \{n_1, n_2, \dots, n_N\}$ is a set of program instructions and N is the total number of instructions. Based on the program analysis framework, we can obtain program text, structure, which is the execution process and data dependency of a program, and attribute information, such as instruction types, opcodes, and operands. We also get a set of entity-relation-entity triples $T = \{t_1, t_2, \dots, t_J\}$ and a set of attributes $A_p = A_{ins} \cup A_{bb}$, where J is the total number of relations, $A_{ins} = \{I_1, I_2, \dots, I_N\}$ presents the attributes of instructions and $A_{bb} = \{b_1, b_2, \dots, b_M\}$ presents the attributes of basic blocks. Each triple t_i is in the form of (C_i, r_i, C_k) , where $C_i, C_k \in BB \cup IN$ and r_i is the relation between the entities C_i and C_k . Therefore, entities in this paper include both instructions (IN) and basic blocks (BB). Relations of instruction entities include *control flow* (execution sequence), *data flow* (data transfer), *jump*, *call*, and *access*. The relation of basic block entities is *control flow*. The relation between the basic block entities and instruction entities is *inclusion*, which indicates that the basic block contains this instruction. Based on those notations, we can provide the definition of the instruction vulnerability and formulate the problem of instruction vulnerability prediction as follows:

Definition 1 (Instruction Vulnerability). Instruction vulnerability is the probability that program results may be incorrect (SDC) raised by the change of an instruction n_i due to bit flip during execution, denoted as y_i . Y is the set of all vulnerabilities of instructions in the program S .

Problem 1 (Instruction Vulnerability Prediction). Given a small set of instruction vulnerability Y_{train} inferred by methods like hardware fault injection for training, instruction vulnerability prediction can be formulated as a semi-supervised learning problem:

$$\{BB, IN, T, A_p, Y_{train}\} \xrightarrow{\mathcal{F}(\cdot)} Y. \quad (2)$$

IV. THE PROPOSED METHOD

To solve the problem of the Instruction Vulnerability Prediction, we first construct a multi-layer heterogeneous program knowledge graph $G = (V, E)$ about the program S to model the complex relations between the sets BB , IN , T , and A_p , which implicates the potential patterns in error propagation. Subsequently, we turn to extract the expressive instruction embedding from this knowledge graph to predict the vulnerability with GNN and transformer.

A. Multi-Layer Heterogeneous Program Knowledge Graph Construction

Fig. 2 is the pipeline of multi-layer heterogeneous program knowledge graph construction in three parts: **1)** We perform

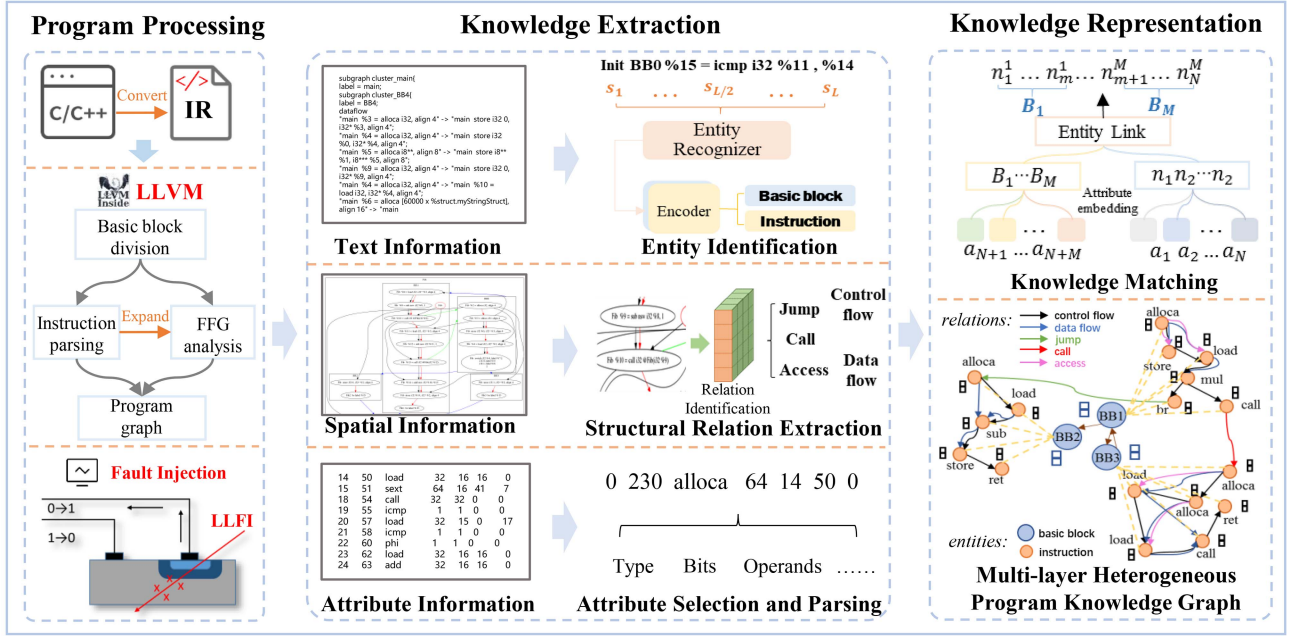


Fig. 2. Framework diagram of multi-layer heterogeneous program knowledge graph construction.

comprehensive program analysis and register-related fault injection to explore the complexity of error propagation through the constructed instruction execution and fault generation system. 2) To address the heterogeneity and complexity of multi-source data, we develop a knowledge extraction module, which integrates both structural and non-structural information and creates a knowledge base for graph construction. 3) A multi-layer heterogeneous knowledge graph is constructed based on the acquired knowledge base, which consists of basic blocks and instruction nodes with complicated structural relations. Then, we will go into detail.

1) *Program Processing*: Through secondary development of the LLVM (a compilation framework for lifelong program analysis and transformation) and LLFI (an intermediate code-level fault injection tool for hardware faults), we develop an instruction execution and register fault generation system to extract program information, which implies error propagation patterns, and build a multi-source database containing semantic, structural, and attribute information. As shown in Algorithm 1, the key steps are as follows:

Basic Block Division: The program S is split into several basic blocks based on the LLVM (line 1), where each basic block is composed of several instructions. Furthermore, we obtain the information contained in each basic block, such as the number of instructions and relations with other basic blocks (lines 4-6).

Instruction Parsing: Each instruction can be expressed as $\{function\ name, basic\ block\ number, instruction\ information\}$ to form text information about a program, where the *instruction information* consists of opcodes, operands, and so on (line 8). In order to enhance the structural information of the program, instructions with flow relation, such as *call* and *jump* are extended to the full-flow graph analysis. *Full-flow Graph Analysis*: The standard flow graph analysis involves obtaining the *control*

flow and *data flow* from each function. To obtain implicit error propagation patterns, we extend it to the complete program and construct a multi-relational program graph including functions, basic blocks, and instructions by considering instructions and basic blocks with flow relations (lines 9-22). Specifically, ① based on the execution process of functions, instructions for each basic block can be obtained. The jumps between basic blocks (i.e., where the '*br*' instruction points to) are defined as the *control flow* relations between them, carving out the loop structure. ② Then, the data transfer (i.e., *data flow* relation) between instructions is obtained based on the operands of each instruction. And the execution order of instructions is defined as the *control flow* relation between instructions. In handling cyclic structures, we do not unfold loops into dynamic iterations; instead, each loop is represented as a *control flow*. This preserves structural completeness without expanding the graph indefinitely. For the '*call*' instruction, we point it to the first instruction of the called function, and the '*ret*' instruction of that function points back to it. Besides, for the '*br*' instruction, we also point it to the first instruction of the target basic block. Here, we show the extracted full-flow graph using the Fibonacci program, as shown in Fig. 3.

Hardware Fault Injection: The LLFI is a highly configurable program hardware fault injector [16]. LLFI performs fault injection at the IR level, which models internal microarchitectural components (e.g., triggers or latches) at a higher level of abstraction. In LLFI, bit flips are injected into program variables, operands, or instruction outputs (e.g., destination registers) at the IR level during dynamic execution. It simulates errors in data-related components (e.g., ALUs or registers) and is commonly used to approximate faults occurring in the processor's execution units. Thus, we inject faults into the operands (i.e., destination and source registers) of IR instructions (lines 26-31).

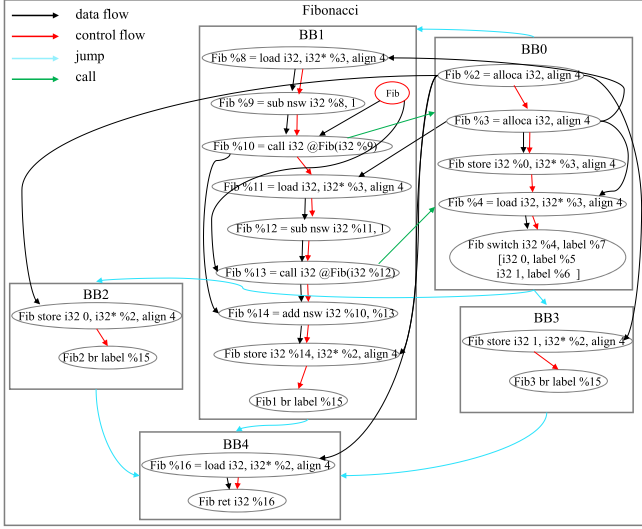


Fig. 3. Full-flow example (Fibonacci.c): The black line indicates *data flow*, the red line indicates *control flow*, the blue line indicates *basic block jump*, and the green line indicates *function call*. Each box represents a basic block.

Based on the fault model, we get the register bits R_I^W of the corresponding instruction from S_e and Reg , and then flip its machine code bit by bit, to realize the effect of random bit flipping during the execution of programs. Finally, a database of faults about the program is formed, and the error rate for each instruction is calculated as follows:

$$P_I = \frac{Num_{err}}{R_I^W \times N^T}, \quad (3)$$

where Num_{err} is the number of SDC errors that occurred.

2) *Knowledge Extraction*: Here, we create a knowledge base based on entities, relations, and attributes, while taking into account the heterogeneity and complexity of multi-source data. As shown in Algorithm 2, the key steps are as follows:

Entity Identification: Textual data is analyzed to identify entities. The string format in the text is as follows: *init BB0 %15 = icmp i32 %11,%14*, where *init* represents the function name, *BB0* represents the basic block number, and *%15 = icmp i32 %11,%14* represents the instruction information, including name and operands, etc. Based on the LLVM instruction set, the entity recognizer and encoder are designed to extract the basic block entities $BB = \{B_1, B_2, \dots, B_M\}$ and the instruction entities $IN = \{n_1, n_2, \dots, n_N\}$.

Structural Relation Extraction: The program's spatial information is analyzed to identify relations between entities and explore possible paths of error propagation. Firstly, according to the extracted basic block entities, the *control flow* relation between basic blocks such as $B_1 \rightarrow B_2 \rightarrow B_4$ is identified. The links between instructions in terms of *control flow*, *data flow*, *jump*, *call*, and *access* are then extracted. In addition, the ownership (*inclusion*) between the basic block and instruction is also represented as a type of entity relation.

Attribute Selection: Entities have a wide variety of attributes, and it is critical for the vulnerability prediction to select key attributes.

Algorithm 1: Static and Dynamic Analysis of the Program.

Input: C/C++ program: S

Output: A set Inf of program information

```

1:  $S' \leftarrow convert\_IR(S)$ 
2: /*Static Analysis*/
3: for each basic block  $B \in F$  do
4:    $Index_B, Ins_B, Presuc_B \leftarrow getInformation(B)$ 
5:    $BCo \leftarrow buildBBControlflow(B, next\_bb(B))$ 
6:    $insertInf(B\_file, \{Index_B, Ins_B, Presuc_B, BCo\})$ 
7:   for each instruction  $I \in B$  do
8:      $Index_I, Sem_I \leftarrow getSemantics(I)$ 
9:     for each operand  $O \in I$  do
10:       $Users \leftarrow getDependency(O)$ 
11:       $Da \leftarrow buildDataflow(I, Users)$ 
12:    end for
13:    if  $isCallInst(I)$  then
14:       $Fun \leftarrow getCalledFunction(I)$ 
15:       $F_I \leftarrow getFirstInstruction(Fun)$ 
16:       $Ca \leftarrow buildCallflow(I, F_I)$ 
17:    end if
18:     $Co \leftarrow buildControlflow(I, next\_ins(I))$ 
19:     $insertInf(I\_file, \{Index_I, Sem_I, Da, Ca, Co\})$ 
20:  end for
21:   $Ju \leftarrow buildJumpflow(I, nextFirstInst(B))$ 
22:   $insertInf(I\_file, \{Ju\})$ 
23: end for
24:  $Inf \leftarrow B\_file, I\_file$ 
25: /*Dynamic Analysis*/
26:  $initialConfig(F^M)$ 
27: for each dynamic instruction  $D_I \in S'_{run}$  do
28:    $Reg \leftarrow defineRegister(D_I)$ 
29:    $Log \leftarrow injectFault(Reg, F^M)$ 
30:    $Opcode, Width, P_I \leftarrow resAnalysis(Log)$ 
31:    $insertInf(Err\_file, \{Opcode, Width, P_I\})$ 
32: end for
33:  $Inf \leftarrow Err\_file$ 
34: return  $Inf$ 

```

- *Instruction Attributes*. Here, we first take the basic features of instructions, such as the *Opcode*, *Operands*, and *Bits*, as attributes. Then, considering the spatial structure of the program, we include the predecessors (*Pred*) and successors (*Succ*) of instructions as attributes as well. Furthermore, we divide instructions into 8 types based on the official LLVM standard. We carry out fault injection to programs in the benchmark based on the fault model (F^M) defined in this paper, and calculate the error rate for each type of instruction, and finally sum-average the result of each program. The concise overview and results are shown in Table II. It can be seen that different types of instructions have different error rates. Among them, instructions involving data and address operations have the highest error rate. When a bit flip occurs, the execution results of these instructions are changed, and the error propagates to various registers with the execution

TABLE II
ERROR PROBABILITY FOR EACH TYPE OF INSTRUCTION (AVERAGE OF BENCHMARKS)

Type	Description	Error rate
memory_op	Operations on system memory such as <i>load, store</i> .	0.3057
terminator_op	These instructions are used to terminate a basic block of the program, such as <i>br, ret</i> .	0.4166
cast_op	Truncate, expand, and convert instruction, such as <i>trunc, zext</i> .	0.1715
comparison_op	These instructions are used to do comparisons of data, such as <i>icmp, fcmp</i> .	0.8056
integer_op	Integer binary operator, such as <i>add, mul</i> .	0.4943
float_op	Floating-point binary operator, such as <i>fsub, fdiv</i> .	0.5041
logical_op	Logical or shift operation, such as <i>and, shl</i> .	0.5923
other_op	Other types of instructions, such as <i>vaarg, select</i> .	0.2102

of subsequent instructions, causing an erroneous jump or program output. To this end, we take the *Type* as one of the instruction attributes. In summary, attributes of instruction i can be expressed as a six-tuple $I_i = \{Opcode, Bits, Operands, Pred, Succ, Type\}$.

- **Basic Block Attributes.** A basic block is a set of instructions with a single entry point and a single exit point. Therefore, by the Algorithm 1, we take the number of instructions (*Num*) contained in the basic block, predecessors (*Pred*), and successors (*Succ*) as attributes. In summary, attributes of basic block j can be expressed as a three-tuple $b_j = \{Num, Pred, Succ\}$.

Attribute Parsing: To determine the characteristics of each entity, the attribute information needs to be divided. The attribute set of basic block entities is $A_b = \{b_1, b_2, \dots, b_M\}$, including the number of instructions, predecessors, and successors. The attribute set of instruction entities is $A_{ins} = \{I_1, I_2, \dots, I_N\}$, including bits, types, operands and so on. Each attribute is encoded by *LabelEncoder* and normalized to get the program attribute set $\{a_1, a_2, \dots, a_{N+M}\}$. The instruction attributes are defined as follows:

$$a_i^j = \frac{I_i^j - \text{Min}(I_1^j, \dots, I_N^j)}{\text{Max}(I_1^j, \dots, I_N^j) - \text{Min}(I_1^j, \dots, I_N^j)}, \quad (4)$$

where $i \in [1, N], j \in [1, L]$, L represents the number of attribute categories. The basic block attributes are defined as follows:

$$a_{N+i}^j = \frac{b_i^j - \text{Min}(b_1^j, \dots, b_M^j)}{\text{Max}(b_1^j, \dots, b_M^j) - \text{Min}(b_1^j, \dots, b_M^j)}, \quad (5)$$

where $i \in [1, M], j \in [1, l]$, l represents the number of basic block attribute categories.

3) **Knowledge Representation:** In order to understand how error propagates, a multi-layer heterogeneous knowledge graph is constructed using the extracted knowledge triples with attribute data.

Knowledge Matching: Attributes are embedded in the corresponding entities. For the basic block $i, i \in [1, M]$, we assign

Algorithm 2: Knowledge Graph Construction.

Input: Output *Inf* of Algorithm 1

Output: A Multi-Layer Heterogeneous Program

Knowledge Graph: G

```

1:  $B_{file}, I_{file}, Err_{file} \leftarrow getfile(Inf)$ 
2: for each line  $L_i \in B_{file}$  do
3:    $BB \leftarrow entityExtract(L_i)$ 
4:    $T_B \leftarrow relationExtract(BB, L_i)$ 
5:    $A_b \leftarrow featureReg(L_i)$ 
6: end for
7: for each line  $L_i \in I_{file}$  do
8:    $IN \leftarrow entityExtract(L_i)$ 
9:    $T_I \leftarrow relationExtract(IN, L_i)$ 
10:  if  $judgeAccess(IN.last)$  then
11:     $T_I \leftarrow getAccessRelation(IN, L_i)$ 
12:  end if
13:   $A_{ins} \leftarrow insFeatureReg(L_i, Err_{file})$ 
14: end for
15: Acquire  $a$  from  $A_b \cup A_{ins}$  using (1) and (2).;
16:  $G \leftarrow constructGraph(BB \cup IN, T_B \cup T_{ins}, a)$ 
17: return  $G$ 
```

$\{a_{N+i}^1, a_{N+i}^2, \dots, a_{N+i}^L\}$ directly to attribute B_i^A to reduce complexity. For the instruction $j, j \in [1, N]$, we use one-hot encoding to convert discrete attributes such as instruction names and types into ordered vectors and assign $\{a_j^1, a_j^2, \dots, a_j^L\}$ directly to the attribute n_j^A for more efficient operation. In addition, we define a mapping function $\mathcal{M}: \mathcal{I} \rightarrow \mathcal{B}$ to match instructions with their corresponding basic blocks.

Knowledge Graph Construction: As indicated in the lower right corner of Fig. 2, the entity-relation-entity triples are built to create a multi-layer heterogeneous knowledge graph about the program based on basic block entities, instruction entities, and relations between entities that were mined. There is a *control flow* relation between basic block entities, which are represented by the blue node. The instruction entities are represented by orange nodes, and the relations between instruction entities include *control flow*, *data flow*, *jump*, *call*, and *access*. Instruction entities and basic block entities are also related.

B. Vulnerability Prediction Model

Here, we develop an instruction vulnerability prediction model for the multi-layer heterogeneous program knowledge graph. The framework is shown in Fig. 4 with two main parts: ① Since different basic block architectures significantly affect SDC errors [34], we use Graph Convolutional Network (*GCN*) [35] to mine the spatial dependencies of basic blocks. Thus, abnormal jumps can be detected based on unusual contextual relations, and then the basic block containing faulty instructions can be identified. ② Then, based on *inclusion* relations, we aggregate the embedding of basic blocks within their respective instructions. With this as input, Heterogeneous Graph Transformer (*HGT*) [36] is used to mine instruction multi-relational spatial information, thereby further exploring which instructions in the basic block are prone to errors.

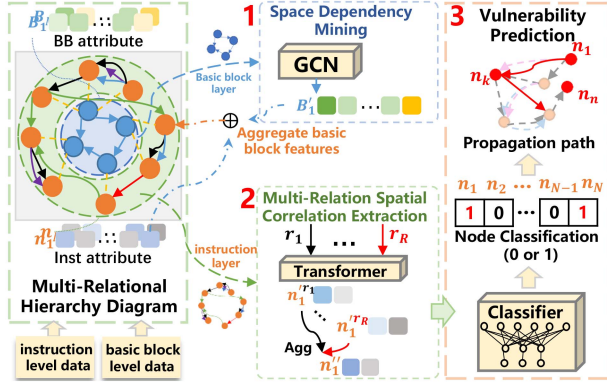


Fig. 4. Framework diagram of instruction vulnerability prediction model.

Let $G = (V, E)$ be an instance of the knowledge graph, V represents the set of nodes, including the attributes of nodes, and E represents the set of edges. And it is associated with the mapping function $\mathcal{Z} : \mathcal{V} \rightarrow \mathcal{R}$ of node types and the mapping function $\mathcal{L} : \mathcal{E} \rightarrow \mathcal{E}$ of edge types. $\mathcal{R}$ and $\mathcal{E}$ are predefined sets of node types and edge types, $|\mathcal{R}| + |\mathcal{E}| \geq 2$. Based on DGL [37], G is represented heterogeneously as the input of subsequent models.

1) *Space Dependency Mining*: From the graph G , the basic block subgraph $G_B = (v_B, e_B)$ is extracted, which contains the structural and attribute information of basic blocks, where $v_B \in V$ represents the set of basic blocks, and $e_B \in E$ represents the set of basic block edges. Then, GCN is used to discover the spatial correlation of basic blocks, defined as follows:

$$B_i^{l+1} = \sigma \left(b^l + \sum_{j \in N(i)} \frac{1}{C_{ji}} B_j^l W^l \right), \quad (6)$$

where $N(i)$ represents the neighbors of the node i , c_{ji} is the product of the square root of node degree, $c_{ji} = \sqrt{|N(j)||N(i)|}$. l represents the number of layers, and σ represents the activation function. Therefore, The basic block embedding updates to $B' = \{B'_1, B'_2, \dots, B'_M\}$, which implies abnormal contextual relations. Then it is transmitted to the instruction layer, where each instruction aggregates the characteristic data of the corresponding basic block. To achieve this, we define an aggregation framework with the following formula:

$$n'_i = \text{Agg}(n_i, \{B'_j | n_i \in B'_j\}), \quad (7)$$

where $i \in [1, N], j \in [1, M]$. Instructions use Agg to combine the data from basic blocks, such as tensor splicing. Finally, we update the original instruction attribute n to $n' = \{n'_1, n'_2, \dots, n'_N\}$.

2) *Multi-Relation Spatial Correlation Extraction*: From the multi-layer heterogeneous program knowledge graph G , the instruction layer subgraph $G_I = (v_I, e_I)$ is extracted, $v_I \in V$ represents the set of instructions, and $e_I \in E$ represents the set of instruction edges. G_I is modeled as a heterogeneous graph consisting of instructions and their many types of relations. Recently, heterogeneous graph neural networks (HGNNs) have

demonstrated significant advantages in handling complex structures and diverse relations [38], [39], [40]. Thus, to effectively capture the rich semantics embedded in such heterogeneous relations, we employ the HGT to extract multiple spatial correlations between instructions. HGT leverages a type-aware transformer to model the error propagation patterns between different types of relations, thereby exploring the location of faulty instructions in basic blocks. Specifically, given the graph G_I and the node attribute n' , we utilize the attention mechanism for each relation (s, e, t) , where $s, t \in v_I$ and $e \in e_I$ is the relation between the nodes s and t , to obtain the attention score $\text{Att}(s, e, t)$, which captures the semantic information of different relations. The formula is as follows:

$$\text{Att}(s, e, t) = \sigma \left((K(s)W_{\varphi(e)}^{ATT}Q(t)^T) \bullet \frac{\mu_{\langle \tau(s), \varphi(e), \tau(t) \rangle}}{\sqrt{d}} \right), \quad (8)$$

where $K(s) = W_K(n'[s])$, $Q(t) = W_Q(n'[t])$. Then we calculate the message passing paradigm $\text{Message}(s, e, t)$ for each edge:

$$\text{Message}(s, e, t) = W_M n'[s] W_{\varphi(e)}^{MSG}. \quad (9)$$

The message is propagated to the target node, which then aggregates the information from all neighbors:

$$\tilde{n}''[t] = \sum_{s \in N(t)} (\text{Att}(s, e, t) \bullet \text{Message}(s, e, t)). \quad (10)$$

Thus, the instruction embedding n'' containing multi-relation spatial dependencies is obtained:

$$n''[t] = W_A(\sigma(\tilde{n}''[t])) + n'[t]. \quad (11)$$

Consequently, we obtain the prediction result y' :

$$y' = \text{MLP}(\text{Relu}(n'')). \quad (12)$$

Here, the cross-entropy loss is defined as:

$$L = - \sum_{i=1}^c y \ln y'. \quad (13)$$

In the end, through the knowledge graph G and prediction result y' , we can deduce the specific location of instructions with high vulnerability in the program. Furthermore, by leveraging the edge attention weights of the model, we can effectively mine critical paths of error propagation. This information can then be utilized to conduct subsequent redundancy analysis and enable us to gain profound insights into the intricate nature of error propagation within the system.

V. EXPERIMENTS AND RESULTS

In this section, we perform comprehensive experiments to validate the efficacy of our VP-HPKG approach. Furthermore, we are very willing to openly share our code and dataset to foster collaborative discussions and encourage further research in the field of instruction vulnerability prediction. The verifiable data and code of the experiments could be fetched in the link: <https://github.com/SINCOSLab/VP-HPKG>.

A. Fault Generation

Recent studies [41], [42] have demonstrated that fault injection at the IR level yields SDC rates that are comparable in accuracy to those obtained through assembly-level fault injection, which has been shown to closely approximate the effects of real-world soft errors. And many state-of-the-art works have adopted LLFI as a practical and reliable method for large-scale SDC assessment [9], [31]. Following these prior works, we redevelop the LLFI tool by implementing a fine-grained bit-flip mechanism to account for error randomness and propagation. This modification enables us to flip the register bits corresponding to each instruction, replicating the random bit-flip effect during program execution.

Based on some works [9], [31], [33], unlike random injection, we adopt a bit-wise fault injection strategy, where each bit of an instruction is systematically flipped to evaluate its vulnerability. Unlike random sampling, which may overlook critical vulnerable bits due to limited coverage, this exhaustive approach eliminates sampling bias and provides comprehensive and reproducible results, enabling accurate assessment of instruction vulnerabilities. Additionally, we randomly select a subset of instructions and apply the bit-wise fault injection strategy 100 times. Our FI results exhibit an error bar of 0.01% at a 95% confidence interval, which matches the recent studies [9], [31], [32]. Throughout the experiment, a bit-level inversion is applied to every program instruction, resulting in a total of 1,144,070 hardware faults injected. The number of fault injections for each program is shown in Table III. Subsequently, each hardware fault is meticulously counted and analyzed for this study.

B. Experimental Setup

1) *Data Sets*: We conduct experiments with 22 programs from the MiBench [43], Siemens [44] and JetStream2 [45] benchmark suits to enforce diversity in the source code implementation, functionality, and domain, which are widely employed in some relevant studies [46], [47], [48], [49]. These programs include Stack (stack operations), Rad2Deg (conversion from radians to degrees), DataManager (data management system), Isqrt (square root of an integer), BFS (breadth first search), Dijkstra, Bitstring (processing of binary data), and others. We utilize *clang* to compile these programs into the LLVM intermediate representation (IR) to facilitate the analysis of the program structure. A concise overview of programs employed in our experiments is provided in Table III.

2) *Set up*: All experiments and evaluations are carried out on a high-performance machine equipped with an Intel Xeon E5-2630v4 CPU (X86-64), 64 GB of running memory, the operating system Ubuntu 14.04, and the compiler version LLVM 4.0. The instruction vulnerability prediction model is implemented in Pytorch-1.10.2 with Adam optimizer. The learning rate is set to 0.01. *LEAKY_RELU* is applied as the activation function. We partition the program instructions into three sets: 70% for training, 10% for validation, and the remaining instructions for testing. We select the one with the best performance in the

TABLE III
STATISTICS OF PROGRAMS STUDIED IN OUR EXPERIMENTS

Suite	Program	Static Instructions	Relations	Dynamic Instructions	Faults Injected
Mibench	Stack	96	211	718	22980
	Rad2deg	56	167	598	19140
	Datamanager	242	628	2908	46530
	Isqrt	89	305	2799	44790
	BFS	171	439	800	25600
	Dijkstra	525	1445	4763	76200
	Bitstring	93	248	4850	77600
	basicmath	206	542	3446	55130
	CRC	289	766	648	10380
	Partial	200	474	703	11240
Siemens	Susan	12619	38071	49104925	160000
	Paq8p	17752	47361	2362985	48950
	Replace	1439	3985	3815	61040
	Schedule	589	1531	5779	92470
	Schedule2	585	1527	4676	74820
JetStream2	Printtoken	938	2498	1600	25600
	Qsort	92	440	960	30720
	Float-mm	168	420	368	11780
	N-body	447	1308	4389	70220
	Towers	267	666	2295078	160000
	Rot	551	1512	858	13728
	Factorial	200	474	161	5150

validation set and then evaluate it on the test set. We run 3000 epochs for the model.

Our method operates at the LLVM IR level, which serves as a language- and architecture-independent intermediate representation. A wide range of programming languages (e.g., C/C++, Rust, Fortran) can be compiled into IR, and conversely, the IR can target multiple instruction set architectures (e.g., x86, ARM, RISC-V) [50], [51]. By modeling instruction vulnerability at the IR level, our method gains strong generalizability across both source languages and hardware backends, making it suitable for diverse software systems. That said, we acknowledge that our reliance on LLVM IR inherently limits direct applicability to ecosystems that do not compile to LLVM, such as Java bytecode and .NET CLR. Adapting our method to such environments would require additional front-end or IR translation layers, which may introduce engineering overhead. We leave these directions as future work.

3) *Comparison Baseline*: We select nine state-of-the-art methods as comparative experiments.

- **Trident** [32], which applies SVM with a radial basis function kernel to predict error-prone instructions.
- **SDIFI** [34], which uses an improved LightGBM model for SDC error detection.
- **GATPS** [14], which uses the program relation graph and the encoding of instructions to predict which instruction errors will result in SDC.

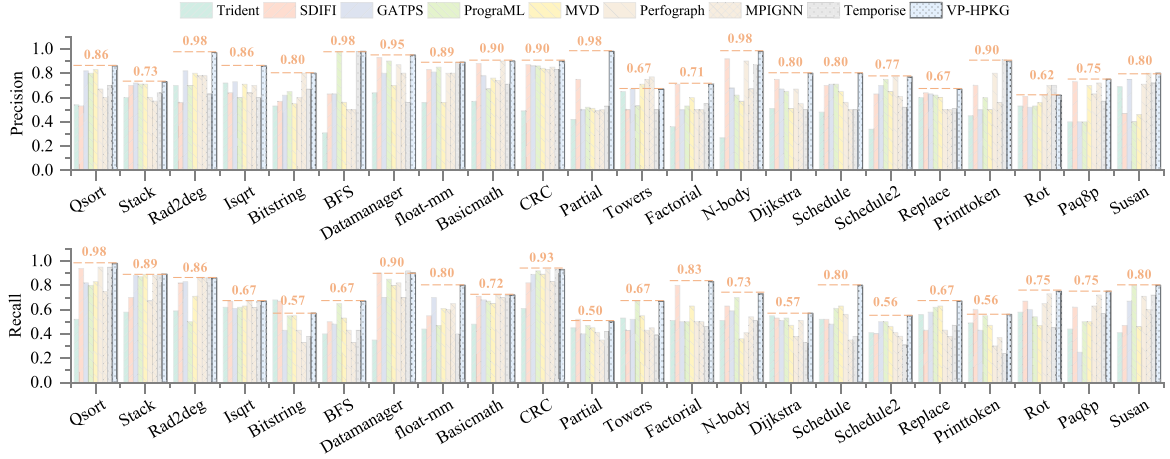


Fig. 5. Performance of different methods in prediction quality for vulnerable instructions.

- **PrograML** [52], which analyzes the SDC by expressing the program at the instruction level, for subsequent downstream tasks.
- **MVD** [53], which uses the *control-data-call* flow graph and program statement to predict memory errors at the source code level.
- **Perfograph** [54], which captures numerical information and data structure by introducing new nodes and edges, and proposes an adapted embedding method to incorporate data awareness at the IR level.
- **MPIGNN** [55], which utilizes IR embeddings and graph attention convolution to tackle the issue of identifying errors in programs.
- **Temporis** [27], which leverages the control data flow graph and calling context tree to represent execution states. Then, the SDC probabilities can be predicted by a graph attention network.

4) *Evaluation Metrics*: The effectiveness of the instruction vulnerability prediction can be evaluated by the following widely-used metrics [56]: accuracy (*Acc*), precision (*Pre*), recall (*Rec*), F1-score (*F1*). The formulas for the calculation of these metrics are as follows:

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}, \quad (14)$$

$$Pre = \frac{TP}{TP + FP}, \quad Rec = \frac{TP}{TP + FN}, \quad (15)$$

$$F1 = \frac{2 \times TP}{2 \times TP + FN + FP}. \quad (16)$$

where *TP* (True Positive) refers to the cases in which the model correctly predicts the vulnerable instructions. *TN* (True Negative) refers to the cases where the model correctly predicts the non-vulnerable instructions. *FP* (False Positive) refers to the cases where the model incorrectly predicts the vulnerable instructions. *FN* (False Negative) refers to the cases where the model incorrectly predicts the non-vulnerable instructions. *Acc* measures the proportion of correctly predicted instructions among all instructions. *Pre* measures the proportion of truly

vulnerable instructions among the predicted vulnerable instructions. *Rec* measures the proportion of predicted vulnerable instructions among the truly vulnerable instructions. *F1* measures the overall effectiveness by considering both precision and true positive rate. These four metrics capture the performance of all methods in various aspects, where higher values close to 1 for these indicators indicate superior predictive performance.

C. Performance of Prediction

We perform experiments on the vulnerability prediction for each of the 22 programs. Table IV and Fig. 5 show the predicted results of VP-HPKG in comparison to other baselines.

① *Our method significantly outperforms the state-of-the-art methods in almost all programs.* In Table IV, VP-HPKG consistently outperforms all baselines across almost all programs (21). Specifically, VP-HPKG exhibits an average accuracy of 0.85 and an average F1 of 0.77. Compared to the most competitive baseline, our method improves 10.3% in *Acc* and 18.4% in *F1*. Additionally, compared with traditional error analysis methods (i.e., Trident), VP-HPKG improves *Acc* and *F1* by 44.1% and 54%, respectively. It can also be shown that our method performs well in programs of varying scales and thus exhibits good generalizability. This superiority can be attributed to the advantage of the proposed knowledge graph and GNNs, which effectively capture both data dependencies and execution relations between instructions, enabling more comprehensive error propagation analysis. Thus, our method can be used to analyse the instruction vulnerability of each module during the stages of software testing and deployment. The developers can thus design protection mechanisms at the IR level based on different task requirements to improve software reliability intelligently with low cost.

② *VP-HPKG demonstrates superior performance in identifying vulnerable instructions.* Based on the prediction values and the error rates obtained by fault injection, the *Pre* and *Rec* are derived for vulnerable instructions, and the results are shown in Fig. 5. It can be seen that our method can more accurately predict error-prone instructions. Specifically, VP-HPKG

TABLE IV
PERFORMANCE COMPARISON OF RESULTS FOR INSTRUCTION VULNERABILITY PREDICTION IN DATASETS

Type	Program	Trident		SDIFI		GATPS		PrograML		MVD		Perfograph		MPIGNN		Temporise		VP-HPKG	
		Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	A	F1	Acc	F1
Small Benchmarks	Qsort	0.58	0.53	0.71	0.53	0.75	0.78	0.76	0.67	0.82	0.76	0.85	0.80	0.70	0.67	0.78	0.67	0.93	0.92
	Stack	0.62	0.59	0.71	0.77	0.68	0.70	0.73	0.76	0.71	0.75	0.61	0.63	0.61	0.69	0.72	0.78	0.78	0.80
	Rad2deg	0.66	0.64	0.56	0.71	0.64	0.78	0.62	0.74	0.77	0.86	0.82	0.87	0.81	0.87	0.63	0.78	0.91	0.92
	Isqrt	0.67	0.66	0.64	0.71	0.68	0.70	0.62	0.63	0.71	0.69	0.71	0.74	0.64	0.73	0.64	0.75	0.76	0.75
	Bitstring	0.60	0.59	0.64	0.61	0.71	0.65	0.68	0.66	0.62	0.59	0.65	0.50	0.71	0.50	0.65	0.54	0.76	0.67
Middle Benchmarks	BFS	0.55	0.35	0.72	0.52	0.71	0.61	0.90	0.62	0.75	0.60	0.85	0.47	0.84	0.49	0.90	0.57	0.95	0.80
	Datamanager	0.56	0.45	0.85	0.87	0.71	0.75	0.87	0.88	0.66	0.67	0.85	0.85	0.76	0.75	0.61	0.72	0.91	0.92
	Float-mm	0.59	0.49	0.79	0.67	0.76	0.75	0.82	0.78	0.63	0.67	0.83	0.75	0.83	0.80	0.75	0.57	0.88	0.84
	Basicmath	0.61	0.53	0.78	0.79	0.77	0.77	0.75	0.79	0.75	0.78	0.75	0.80	0.78	0.80	0.75	0.80	0.78	0.80
	CRC	0.60	0.54	0.85	0.91	0.80	0.88	0.82	0.89	0.77	0.86	0.82	0.90	0.74	0.84	0.82	0.90	0.86	0.92
	Partial	0.56	0.44	0.90	0.65	0.89	0.50	0.96	0.51	0.88	0.53	0.88	0.49	0.89	0.52	0.89	0.52	0.96	0.67
	Towers	0.63	0.58	0.76	0.46	0.73	0.43	0.78	0.49	0.79	0.62	0.80	0.57	0.80	0.50	0.70	0.51	0.80	0.67
	Factorial	0.57	0.42	0.75	0.75	0.57	0.50	0.57	0.40	0.69	0.61	0.57	0.50	0.57	0.50	0.43	0.42	0.79	0.77
Large Benchmarks	N-body	0.55	0.35	0.82	0.68	0.76	0.63	0.81	0.53	0.68	0.44	0.78	0.56	0.75	0.60	0.75	0.46	0.91	0.84
	Dijkstra	0.60	0.53	0.78	0.50	0.75	0.47	0.81	0.51	0.76	0.45	0.84	0.43	0.73	0.40	0.81	0.49	0.89	0.67
	Schedule	0.59	0.50	0.75	0.58	0.82	0.68	0.83	0.69	0.76	0.62	0.80	0.51	0.84	0.53	0.80	0.51	0.91	0.80
	Schedule2	0.55	0.37	0.71	0.53	0.71	0.62	0.65	0.54	0.63	0.61	0.63	0.49	0.58	0.51	0.74	0.58	0.76	0.65
	Replace	0.62	0.58	0.65	0.60	0.75	0.59	0.84	0.63	0.78	0.50	0.82	0.50	0.77	0.54	0.77	0.53	0.89	0.67
	Printtoken	0.58	0.47	0.80	0.68	0.71	0.50	0.85	0.54	0.71	0.49	0.78	0.52	0.67	0.53	0.72	0.48	0.87	0.69
	Rot	0.60	0.55	0.77	0.66	0.68	0.55	0.73	0.48	0.65	0.51	0.77	0.68	0.78	0.70	0.67	0.56	0.77	0.68
	Paq8p	0.56	0.42	0.86	0.71	0.70	0.31	0.83	0.44	0.83	0.61	0.73	0.63	0.75	0.72	0.72	0.57	0.87	0.75
Average	Susan	0.63	0.51	0.70	0.47	0.85	0.74	0.85	0.57	0.45	0.46	0.75	0.71	0.90	0.75	0.77	0.72	0.90	0.80
	Average	0.59	0.50	0.75	0.65	0.73	0.63	0.77	0.63	0.71	0.62	0.76	0.63	0.74	0.63	0.73	0.61	0.85	0.77

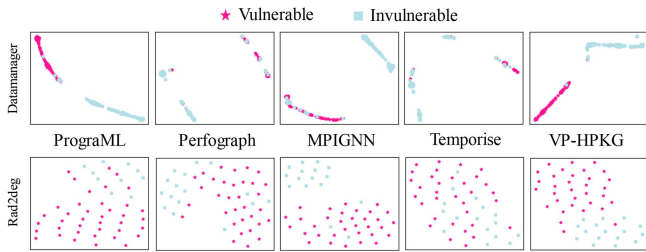


Fig. 6. T-SNE visualization diagram of the final layer instruction representation of different methods on Datamanager (top line) and Rad2deg programs (bottom line). Instruction colors and shapes indicate different result classes.

exhibits an average precision of 0.83 and an average recall of 0.74. The average precision is 21.2% higher than Perfograph and 30.5% higher than MVD. The average recall is 16.6% higher than PrograML and 31.6% higher than Temporise. These results demonstrate that VP-HPKG effectively identifies error-prone instructions with great accuracy. Besides, we use T-SNE [57] visualization to intuitively evaluate the prediction results of the model's classification performance. Fig. 6 displays the embedded visualization of the DataManager and Rad2Deg programs.

In the visualization, different categories are represented by node shapes and colors, with pink star nodes specifically indicating high-vulnerability instructions. It can be seen that points of the same category tend to converge in a group. Compared with baselines, our method can learn instruction embedding better and predict instruction vulnerabilities effectively.

D. Efficiency Analysis

1) *Model Complexity*: We first evaluate the model size, FLOPs (Floating Point Operations), and execution time for the VP-HPKG and baselines to analyze the space-time overhead of these methods. The model size represents the number of model parameters and measures the memory overhead. The FLOPs represent the number of floating point operations (such as additions and multiplications), which is commonly used to assess the computational complexity of models in the field of deep learning [58]. Table V presents the results of this evaluation. It can be seen that VP-HPKG achieves the most compact architecture, with only 104 parameters and 71.9 k FLOPs. The model size and computational complexity (FLOPs) of our method are both reduced by about 2~44 times compared to baselines. Furthermore, although the average execution time

TABLE V

COMPARISON OF MODEL COMPLEXITY. 'MODEL SIZE' REPRESENTS THE NUMBER OF MODEL PARAMETERS. 'FLOPS' REPRESENTS THE NUMBER OF FLOATING POINT OPERATIONS.

Method	Model size (params)	FLOPs	Execution time (s)
GATPS	4498	3063600	63.45
PrograML	549	378024	43.65
MVD	732	455595	46.32
Perfograph	207	135792	30.95
MPIGNN	4498	4519835	75.70
Temporise	514	352347	31.88
VP-HPKG	104	71882	44.42

of Perfograph is 13.47s less than our method, it is negligible considering the memory and computational overheads. Thus, combining Tables IV and V, it is evident that our method can predict instruction vulnerability more efficiently, making it particularly suitable for resource-constrained and real-time dependable computing environments.

2) *Efficiency of Fault Injection*: Traditional machine learning methods typically rely on a substantial number of hardware fault injections to acquire an adequate amount of training samples, enabling the model to achieve the desired effectiveness. In contrast, our method requires hardware fault injection for only a subset of program instructions to obtain categories. The model then leverages the multi-layer heterogeneous knowledge graph of the program to deduce the categories of remaining instructions, resulting in a reduction of necessary hardware fault injections. Fig. 7 shows the performance of our model under various hardware fault injection ratios. It can be seen that VP-HPKG outperforms the most competitive methods in most cases and achieves better prediction performance with fewer fault injection ratios. This demonstrates that our method can tolerate a certain amount of data missing, and the performance overhead will be smaller. For example, the prediction effect of our method at a 30% fault injection ratio is similar to that of MPIGNN at a 70% fault injection ratio. Simultaneously, by injecting hardware faults into 70% of program instructions, we achieved a prediction accuracy of 0.85. Therefore, our method can reduce fault injection costs by 30% while ensuring prediction accuracy.

E. Ablation Study

1) *Contributions of Different Modules*: We construct a multi-layer heterogeneous program knowledge graph and leverage the GNN and Transformer to predict error-prone instructions. Our knowledge graph comprises a homogeneous basic block layer and a heterogeneous instruction layer, which captures the pattern of error propagation. In order to verify the effectiveness of these components, several variants of VP-HPKG are introduced for other comparisons:

- VP-HPKG-nbb, where the basic block layer is removed;
- VP-HPKG-nac, where the *access* relation is removed;
- VP-HPKG-nk, where the *control flow* is removed;

TABLE VI

ABLATION RESULTS ON DIFFERENT COMPONENTS

Component	Acc	Pre	Rec	F1
VP-HPKG-nbb	0.823	0.808	0.591	0.676
VP-HPKG-nac	0.765	0.777	0.562	0.645
VP-HPKG-nk	0.830	0.741	0.597	0.655
VP-HPKG-nc	0.846	0.796	0.604	0.681
VP-HPKG-nd	0.804	0.728	0.643	0.677
VP-HPKG	0.856	0.831	0.744	0.773

- VP-HPKG-nc, where the *call* relation is removed;
- VP-HPKG-nd, where the *data flow* is removed.

For each ablation, we train the model from scratch using an equivalent experimental setup while varying individual components. The results are shown in Table VI.

① *Multi-Layer Structure*. It can be seen from Table VI that the predicted effect of VP-HPKG-nbb is significantly reduced. F1 score decreased by 12.7%, accuracy and precision decreased by 3.6% and 3%, respectively. This is because the attributes of basic blocks have a certain impact on the vulnerabilities of instructions. Additionally, vulnerable instructions can be more precisely located based on abnormal basic block jumps. Basic blocks that are prone to errors are often composed of some vulnerable instructions. The experiment shows that the multi-layer structure information of the program has a certain impact on the instruction vulnerability.

② *Multiple Relations Between Instructions*. The effect of VP-HPKG-nac is most pronounced since load and store instructions are highly dependent on memory and registers. Erroneous data stored in registers or memory, as well as errors occurring in memory or registers and subsequently loaded into specific instructions, can potentially impact the subsequent program execution to varying extents. Besides, the *control flow*, *data flow*, and *call* relations represent the fundamental flow relations within a program. Without any of them, the structural information of the program is incomplete, which will affect the method's error reasoning ability, resulting in worse prediction performance.

2) *Data Efficiency*: To evaluate the data efficiency and the impact of fault injection ratio, we vary the training set size from 10% to 100% of 70% instructions, while keeping the validation and test sets unchanged, the results are as shown in Fig. 8. It can be seen that the overall test performance improves as the proportion of training data increases, indicating that the model can leverage more training samples to better capture instruction context. And our method achieves good accuracy at relatively low training sets (e.g., the average accuracy reaches 77% with a 30% training set), exhibiting that our method has high data efficiency and does not lose much performance at low fault injection. These results validate its potential for deployment in scenarios with limited simulation resources. Besides, our method adopts an offline training paradigm, in which fault injection is performed before program deployment (i.e., before deployment, developers can identify potentially vulnerable instructions based on our model). In this way, our method strikes a practical balance

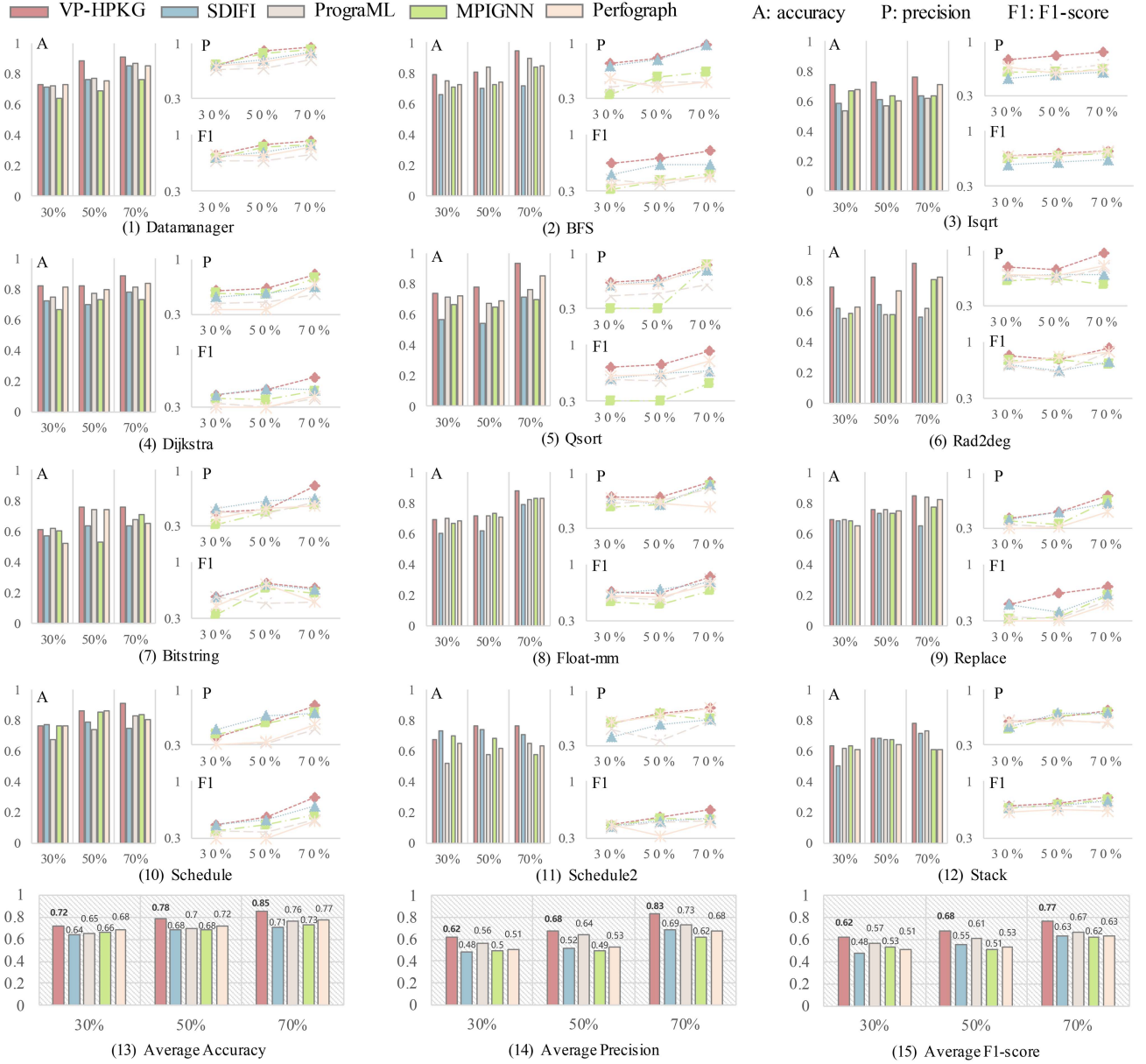


Fig. 7. Performance of different fault injection ratios. 30%, 50%, and 70% of instructions are randomly selected for fault injection to train the model, and then the remaining instructions are used to evaluate the prediction performance.

between data efficiency and real-world applicability, offering an effective solution for early-stage fault tolerance enhancement.

F. Error Propagation Analysis

1) *Error Propagation in Control and Data Flow*: We utilize visualization to analyze how error propagates by examining the learned edge weights of models. In Fig. 9, we present the representation of edge weights learned by our model for the Qsort program. The color depth of each edge reflects the normalized weight coefficient of the aggregation between instructions. Darker colors indicate greater weight values, suggesting a stronger influence on adjacent nodes and a higher likelihood of backward propagation along the edge. From Fig. 9, it is evident that our method surpasses PrograML in scenarios involving the

simple *control flow* relation. VP-HPKG demonstrates superior accuracy in analyzing critical propagation paths of error when the program is executed sequentially. For instance, in the *control flow*, when No.12 instruction calls the *strcmp* for string comparison, the resulting value is either 0 or 1. And only one bit in "%14" is required to store this result, and when that bit is flipped, it will affect the execution of the subsequent program. Thus, there should be a weak correlation between No.12 and No.13. However, PrograML believes that there should be a strong correlation between them, which is inconsistent with the fact. Besides, in scenarios where there are simple *data flow* relations, VP-HPKG also demonstrates superior performance. The essential propagation path of the error in the *data flow* can also be more precisely determined. For example, the No.44 instruction in Fig. 9 calls the *fopen* to open the file, the "%19"

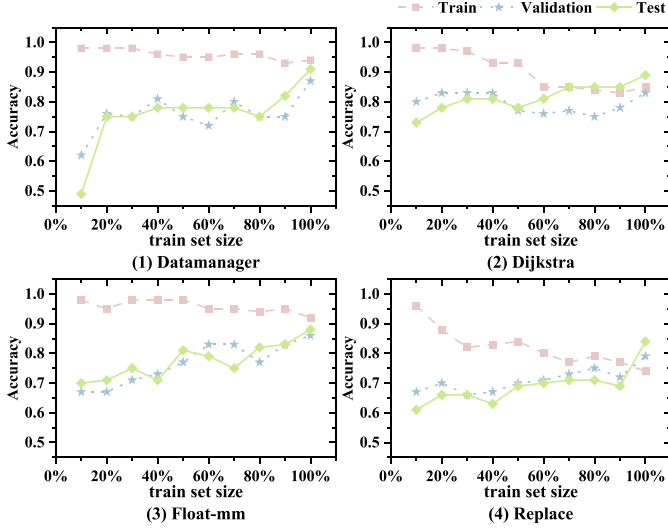


Fig. 8. Learning curves on instruction vulnerability prediction. Accuracy is measured on train, validation, and test sets under varying train set sizes (10 100%) across four programs: (1) Datamanager, (2) Dijkstra, (3) Float-mm, and (4) Replace.

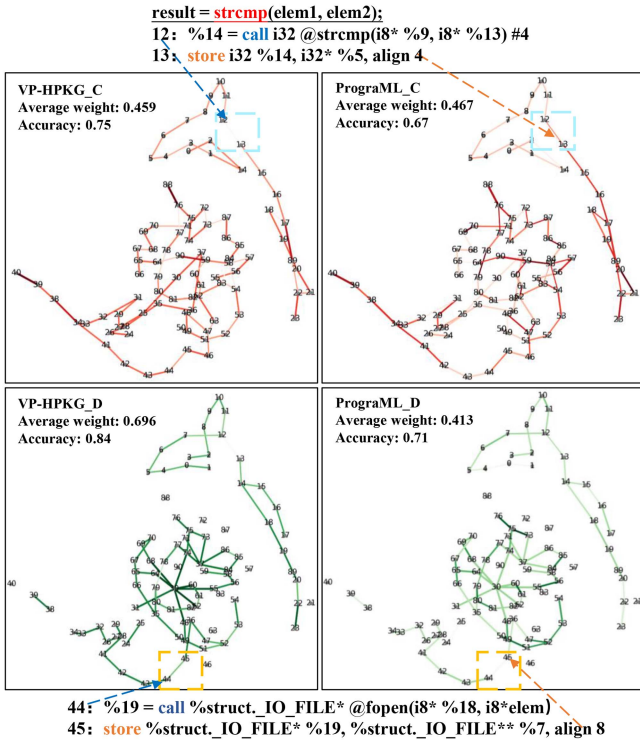


Fig. 9. The edge weights learned by the model on the Qsort program. Left: The edge weights learned by VP-HPKG. Right: The edge weights learned by PrograML. Top: Weights of the control flow relation. Bottom: Weights of the data flow relation.

stores the pointer to the file or null. When the bit flip occurs, the pointer to read and write the file changes, which affects the running result of the program. The PrograML, however, believes that this will not result in errors. In a computer system, errors typically originate in registers or memory units, leading to incorrect data being retrieved and subsequently propagated backward

TABLE VII
PERFORMANCE COMPARISON ON MULTI-THREAD PROGRAMS
(I.E., STRING_MATCH)

Methods	Accuracy	Precision	Recall	F1-score
SDIFI	0.74	0.78	0.85	0.81
GATPS	0.74	0.76	0.83	0.79
PrograML	0.72	0.77	0.83	0.80
MVD	0.75	0.82	0.85	0.83
Perfograph	0.75	0.75	0.88	0.81
MPIGNN	0.71	0.75	0.83	0.78
Temporise	0.75	0.75	0.87	0.80
VP-HPKG	0.79	0.85	0.88	0.86

as directed by the instructions. However, it is important to note that data is not typically accessed in the exact order of program execution, but only the instructions that utilize the data receive it. As a result, errors are often propagated backward with the execution of the *data flow*.

2) *Analysis of Multithreaded Programs*: While our method primarily leverages static control and data flow analysis at the IR instruction level, it is important to emphasize that multithreaded programs, at their core, still consist of instructions that execute and propagate data or errors in a sequential fashion within each thread. VP-HPKG essentially models error propagation paths across threads by capturing all possible data dependencies, regardless of dynamic scheduling. Thread interactions (e.g., shared memory accesses) are explicitly abstracted as data flow edges in our method, enabling propagation patterns to be identified without requiring thread-aware analysis. Thus, our method can generalize across different concurrency scenarios without requiring thread-specific modeling or dynamic runtime tracing. To validate this, we perform instruction vulnerability prediction on representative multithreaded programs (i.e., string_match) from the Phoenix benchmark [59], as shown in Table VII. Compared to the most competitive baseline, our method improves 5.3% in accuracy and 3.6% in F1, indicating that VP-HPKG effectively captures the error propagation patterns in multithreaded contexts. Thus, while our framework does not explicitly model dynamic thread interactions, it captures the core semantics of fault propagation through instructions, which is sufficient for vulnerability prediction and mitigation guidance.

G. Location of Vulnerable Instructions

To further validate the effectiveness of VP-HPKG, we compare the predicted instruction vulnerabilities with real fault injection results (ground truth obtained by LLFI), covering different instruction types and program contexts. The result is shown in Fig. 10.

① *VP-HPKG can accurately approximate real instruction vulnerability to SDCs*. It can be seen that our method achieves impressive prediction accuracy across different instruction types. Even for instructions (i.e., *br* and *icmp*), the predicted vulnerabilities are extremely close to ground truths (99%). Besides, we find the vulnerability of the same instruction varies across different positions, such as the *add* instruction in Datamanager. As shown at the top of Fig. 11, instruction No.30

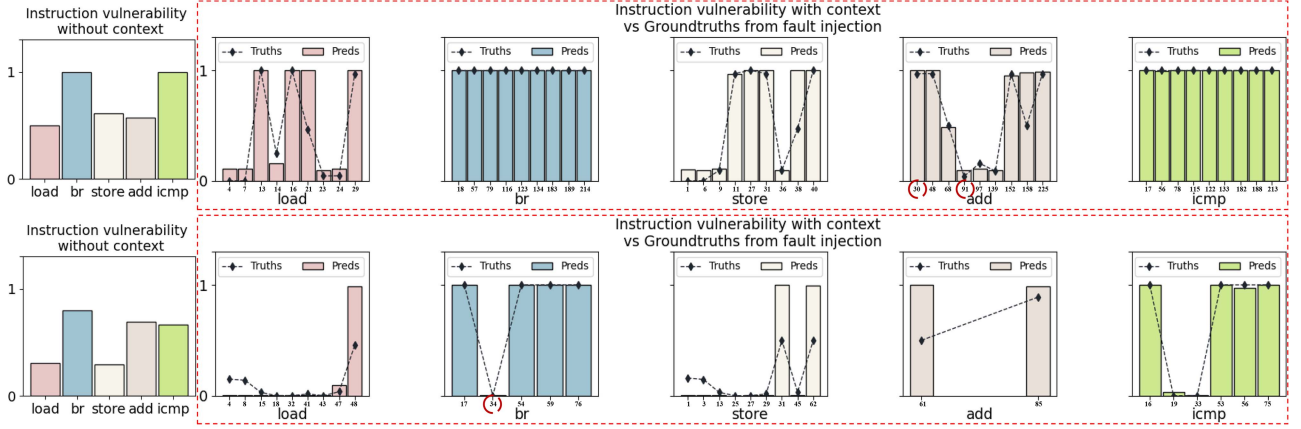


Fig. 10. Predicted instruction vulnerabilities vs ground truths from fault injection. Top: Vulnerability location of the Datamanager program. Bottom: Vulnerability location of the Qsort program.

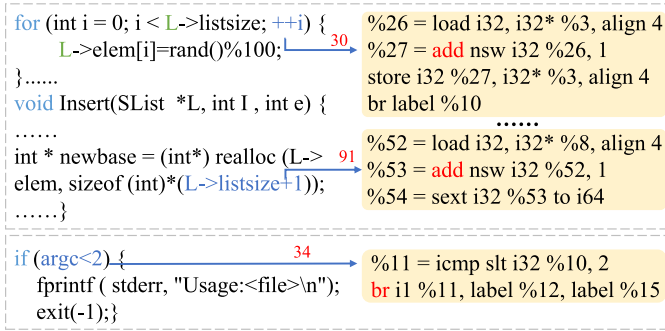


Fig. 11. Part of the source program and corresponding instructions. Top: DataManager. Bottom: Qsort.

implements the operation $i + 1$, which will cause an error in the data stored in %27 when a fault occurs. And with program runs, the error will propagate, causing changes to the *for* loop and ultimately leading to data errors. However, instruction No.91 implements the expansion space, and only a few bits are needed in the register to store the space size because of the small input. Thus, after the bit flip occurs, the possibility of reducing the allocated space is very low, so it will not affect the execution of subsequent instructions. Traditional methods merely provide an overall vulnerability assessment for each instruction, without accurately determining their specific positions within the program. Therefore, the redundancy of all instructions will waste a lot of resources. Our method demonstrates superior accuracy in predicting instruction vulnerability across different contexts, enabling efficient identification of high-vulnerability positions in the program where redundancy can be implemented to minimize costs.

② *The same instruction has different vulnerabilities in different programs.* As shown in Fig. 10, in Datamanager, the instruction *br* is vulnerable in different contexts, while in Qsort, certain positions are not prone to errors. The *br* jumps to different program locations based on conditions. When bit flips occur, the running sequence of the program will change, which may lead to data or logic errors during error propagation. Nonetheless,

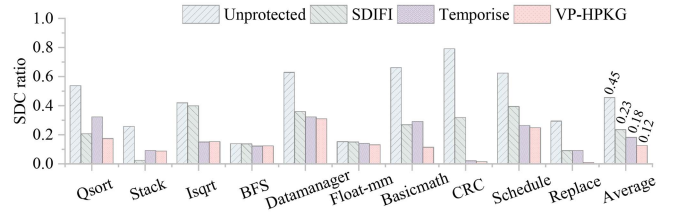


Fig. 12. Comparison of SDC rates before and after applying instruction duplication to 20 vulnerable instructions identified by SIDFI, Temporise, and VP-HPKG, across 10 programs.

there are also certain exceptional cases to consider. As shown at the bottom of Fig. 11, No.34 *br* changes depending on the value of %11. In the source program, for an input with *argc* greater than 2, when an error occurs in the *br*, it will not be propagated backwards, but a program interruption will occur and be detected by the system.

Overall, our method can identify instruction vulnerabilities at various locations in a program, allowing errors to be localized, detected, and hardened.

H. Selective Hardening Validation

To further evaluate the practical utility of VP-HPKG in guiding resilience strategies, we conduct a selective hardening experiment [29], [60] to assess whether the instructions identified as highly vulnerable by our method are indeed critical in real-world SDC propagation. We select 20 vulnerable instructions predicted by some methods (i.e., SDIFI, Temporise, and VP-HPKG) for 10 programs and apply instruction duplication as a mitigation technique. Subsequently, we perform fault injection campaigns on the hardened and original programs under the same injection settings (i.e., 1000 random fault injections). The results are shown in Fig. 12.

Compared to unprotected programs, selectively hardening 20 vulnerable instructions predicted by VP-HPKG results in an average reduction of 72.1% in SDC rates across the evaluated

programs. For instance, the SDC rate of the Datamanager program decreased from 0.629 to 0.311. Even for some programs (i.e., CRC and Replace), the SDC rate has decreased to almost 0 (0.792→0.016 and 0.294→0.01). Besides, it can also be seen that our method is superior to others in most programs. Specifically, compared to the most competitive baseline, VP-HPKG reduces the average SDC rate by 33%. These results demonstrate that the instructions identified by VP-HPKG not only exhibit high vulnerability but also correspond to instructions that play critical roles in actual SDC propagation. This confirms the effectiveness of our method in providing actionable insights for selective fault mitigation.

VI. CONCLUSION AND FUTURE WORK

In this paper, we proposed an efficient instruction vulnerability prediction paradigm: VP-HPKG, which is suitable for various computer systems, like embedded systems or servers. Specifically, first, we built an instruction execution and register fault generation system, based on which we can simulate SDC errors to obtain program semantic information and execution status. Next, due to complex inter-instruction relations and random error propagation, we constructed a multi-layer heterogeneous program knowledge graph by exploiting both the program structural and non-structural information, which implies the potential pattern of propagation. Then, we model the contextual correlations within and across basic blocks from the knowledge graph, with the goal of detecting abnormal inter-block jumps and extracting the instruction embedding to accurately predict vulnerable instructions. Finally, the multi-dimensional experimental results on the benchmarks demonstrated the effectiveness and efficiency of our method.

In future work, we will further study the vulnerability prediction and error detection of computer systems like servers from the perspective of the instruction stream more comprehensively and deeply. In this paper, we focus on SDC errors that cannot be detected by the operating system in real-time and have complex error propagation patterns. Actually, these errors are difficult to predict and detect effectively due to their complex causes, diverse error forms, and random error propagation paths. Therefore, in the subsequent research, we will continue to exploit the impact of different components and different error types on the instruction stream under different architectures, by learning the key implicit features and error propagation paths of the system operation in a data-driven and knowledge-discovery manner, so as to perform system vulnerability prediction and error detection more effectively.

REFERENCES

- [1] A. Martinez-Alvarez, S. Cuenca-Asensi, F. Restrepo-Calle, F. R. Palomo Pinto, H. Guzman-Miranda, and M. A. Aguirre, "Compiler-directed soft error mitigation for embedded systems," *IEEE Trans. Dependable Secure Comput.*, vol. 9, no. 2, pp. 159–172, Mar./Apr. 2012.
- [2] F. Cerveira, R. Barbosa, H. Madeira, and F. Araujo, "The effects of soft errors and mitigation strategies for virtualization servers," *IEEE Trans. Cloud Comput.*, vol. 10, no. 2, pp. 1065–1081, Apr.–Jun. 2022.
- [3] F. Wang and V. Agrawal, "Soft error considerations for computer web servers," in *Proc. 42nd Southeastern Symp. Syst. Theory*, 2010, pp. 269–274.
- [4] F. Previlon, C. Kalra, D. Tiwari, and D. Kaeli, "Characterizing and exploiting soft error vulnerability phase behavior in GPU applications," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 288–300, Jan./Feb. 2022.
- [5] H. Li, C. Shao, and Z. Wang, "Detecting fault injection attacks based on compressed sensing and integer linear programming," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 3, pp. 476–483, May/Jun. 2019.
- [6] A. Martínez-Álvarez, F. Restrepo-Calle, S. Cuenca-Asensi, L. M. Reyneri, A. Lindoso, and L. Entrena, "A hardware-software approach for on-line soft error mitigation in interrupt-driven applications," *IEEE Trans. Dependable Secure Comput.*, vol. 13, no. 4, pp. 502–508, Jul./Aug. 2016.
- [7] M. Didehban, H. So, P. Gali, A. Shrivastava, and K. Lee, "Generic soft error data and control flow error detection by instruction duplication," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 78–92, Jan./Feb. 2024.
- [8] A. R. Anwer, G. Li, K. Pattabiraman, M. Sullivan, T. Tsai, and S. K. S. Hari, "GPU-Trident: Efficient modeling of error propagation in GPU programs," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, pp. 1–15.
- [9] Y. Huang, S. Guo, S. Di, G. Li, and F. Cappello, "Mitigating silent data corruptions in HPC applications across multiple program inputs," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 1–14.
- [10] M. A. Elmohr, H. Liao, and C. H. Gebotys, "EM fault injection on ARM and RISC-V," in *Proc. 21st Int. Symp. Qual. Electron. Des.*, 2020, pp. 206–212.
- [11] V. Bandeira, F. Rosa, R. Reis, and L. Ost, "Non-intrusive fault injection techniques for efficient soft error vulnerability analysis," in *Proc. IFIP/IEEE 27th Int. Conf. Very Large Scale Integration*, 2019, pp. 123–128.
- [12] S. K. Pandey, R. B. Mishra, and A. K. Tripathi, "Machine learning based methods for software fault prediction: A survey," *Expert Syst. Appl.*, vol. 172, 2021, Art. no. 114595.
- [13] A. R. Anwer, G. Li, K. Pattabiraman, M. Sullivan, T. Tsai, and S. K. S. Hari, "GPU-trident: Efficient modeling of error propagation in GPU programs," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, pp. 1–15.
- [14] J. Ma, Z. Duan, and L. Tang, "GATPS: An attention-based graph neural network for predicting SDC-causing instructions," in *Proc. IEEE 39th VLSI Test Symp.*, 2021, pp. 1–7.
- [15] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim.*, 2004, pp. 75–86.
- [16] Q. Lu, M. Farahani, J. Wei, A. Thomas, and K. Pattabiraman, "LLFli: An intermediate code-level fault injection tool for hardware faults," in *Proc. IEEE Int. Conf. Softw. Qual. Rel. Secur.*, 2015, pp. 11–16.
- [17] Y. Yuan and D. Chiou, "Accelerating fault injection for validating processor RTL implementations," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2024, pp. 1–9.
- [18] K. Joshi, R. Singh, T. Bassetto, S. Adve, D. Marinov, and S. Misailovic, "Fastflip: Compositional SDC resiliency analysis," in *Proc. 23rd ACM/IEEE Int. Symp. Code Gener. Optim.*, 2025, pp. 362–376.
- [19] S. Hukerikar, A. Lotfi, Y. Huang, J. Campbell, and N. Saxena, "Optimizing large-scale fault injection experiments through martingale hypothesis: A systematic approach for reliability assessment of safety-critical systems," in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2024, pp. 111–117.
- [20] F. F. dos Santos, J. E. R. Condia, L. Carro, M. S. Reorda, and P. Rech, "Revealing GPUs vulnerabilities by combining register-transfer and software-level fault injection," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2021, pp. 292–304.
- [21] J. Tan, X. Li, A. Zhong, K. Yan, X. Wei, and G. Li, "GEREM: Fast and precise error resilience assessment for GPU microarchitectures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 5, pp. 1011–1024, May 2025.
- [22] Z. Yan, Y. Zhuang, W. Zheng, and J. Gu, "Multi-bit data flow error detection method based on SDC vulnerability analysis," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 3, pp. 1–30, 2023.
- [23] H. Jiang et al., "HAppA: A modular platform for HPC application resilience analysis with LLMs embedded," in *Proc. 43rd Int. Symp. Reliable Distrib. Syst.*, 2024, pp. 40–51.
- [24] J. Jiao, D. Pal, C. Deng, and Z. Zhang, "GLAIVE: Graph learning assisted instruction vulnerability estimation," in *Proc. Des. Automat. Test Europe Conf. Exhib.*, 2021, pp. 82–87.
- [25] X. Wei, J. Zhao, N. Jiang, and H. Yue, "GLAM-SERP: Building a graph learning-assisted model for soft error resilience prediction in GPGPUs," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2023, pp. 419–435.

- [26] P. Yu, J. Gu, D. Shen, X. Dong, Y. Liu, and H. Xiong, "Instruction semantics enhanced dual-flow graph model for GPU error resilience prediction," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2025, pp. 2803–2814.
- [27] J. Ma, Y. Ding, S. Huang, Z. Duan, and L. Tang, "Temporis: Extracting semantic representations of varied input executions for silent data corruption evaluation," *Future Gener. Comput. Syst.*, vol. 166, 2025, Art. no. 107734.
- [28] H. A. haj Ahmad and Y. Sedaghat, "BiGRESi: Robust bit-level fault injection framework for assessing intrinsic software resilience against soft errors," *Comput. Elect. Eng.*, vol. 116, 2024, Art. no. 109222.
- [29] Al-haj Hussien Ahmad and Y. Sedaghat, "An automated framework for selectively tolerating SDC errors based on rigorous instruction-level vulnerability assessment," *Future Gener. Comput. Syst.*, vol. 157, pp. 392–407, 2024.
- [30] H. Cho, S. Mirkhani, C.-Y. Cher, J. A. Abraham, and S. Mitra, "Quantitative evaluation of soft error injection techniques for robust system design," in *Proc. 50th Annu. Des. Automat. Conf.*, 2013, pp. 1–10.
- [31] M. H. Rahman, S. Di, S. Guo, X. Lu, G. Li, and F. Cappello, "Druto: Upper-bounding silent data corruption vulnerability in GPU applications," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2024, pp. 582–594.
- [32] G. Li and K. Pattabiraman, "Modeling input-dependent error propagation in programs," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2018, pp. 279–290.
- [33] M. H. Rahman, A. Shamji, S. Guo, and G. Li, "PEPPA-X: Finding program test inputs to bound silent data corruption vulnerability in HPC applications," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, pp. 1–13.
- [34] W. Fang, J. Gu, Z. Yan, and Q. Wang, "SDC error detection by exploring the importance of instruction features," in *Proc. Int. Conf. Wireless Algorithms, Syst. Appl.*, 2021, pp. 351–363.
- [35] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [36] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proc. Web Conf. 2020*, pp. 2704–2710.
- [37] M. Y. Wang, "Deep graph library: Towards efficient and scalable deep learning on graphs," in *Proc. ICLR Workshop Representation Learn. Graphs Manifolds*, 2019, pp. 1–7.
- [38] Y. Ma, N. Yan, J. Li, M. Mortazavi, and N. V. Chawla, "HetGPT: Harnessing the power of prompt tuning in pre-trained heterogeneous graph neural networks," in *Proc. ACM Web Conf.*, New York, NY, USA, Association for Computing Machinery, 2024, pp. 1015–1023.
- [39] J. Tang et al., "HiGPT: Heterogeneous graph language model," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA, Association for Computing Machinery, 2024, pp. 2842–2853.
- [40] J. Zhang, W. Zeng, J. Tang, and X. Zhao, "Hyperedge graph contrastive learning," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 8502–8514, Dec. 2024.
- [41] L. Palazzi, G. Li, B. Fang, and K. Pattabiraman, "Improving the accuracy of IR-level fault injection," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 243–258, Jan./Feb. 2022.
- [42] L. Palazzi, G. Li, B. Fang, and K. Pattabiraman, "A tale of two injectors: End-to-end comparison of IR-level and assembly-level fault injection," in *Proc. IEEE 30th Int. Symp. Softw. Rel. Eng.*, 2019, pp. 151–162.
- [43] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. 4th Annu. IEEE Int. Workshop Workload Characterization*, 2001, pp. 3–14.
- [44] H. Do, S. Elbaum, and G. Rothermel, "Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact," *Empirical Softw. Eng.*, vol. 10, pp. 405–435, 2005.
- [45] D. Y. Hancock et al., "Jetstream2: Accelerating cloud computing via Jetstream," in *Proc. Pract. Experience Adv. Res. Comput.*, 2021, pp. 1–8.
- [46] P. R. Bodmann, G. Papadimitriou, R. L. R. Junior, D. Gizopoulos, and P. Rech, "Soft error effects on arm microprocessors: Early estimations versus chip measurements," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2358–2369, Oct. 2022.
- [47] B. Sangchoolie, K. Pattabiraman, and J. Karlsson, "An empirical study of the impact of single and multiple bit-flip errors in programs," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1988–2006, May/Jun. 2022.
- [48] G. Papadimitriou and D. Gizopoulos, "Silent data corruptions: Microarchitectural perspectives," *IEEE Trans. Comput.*, vol. 72, no. 11, pp. 3072–3085, Nov. 2023.
- [49] H. Tekgul and O. Ozturk, "Instruction-level reliability improvement for embedded systems," in *Proc. IEEE Int. Conf. Des. Test Integr. Micro Nano-Syst.*, 2020, pp. 1–5.
- [50] W. Moses and V. Churavy, "Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 12472–12485.
- [51] W. S. Moses et al., "Scalable automatic differentiation of multiple parallel paradigms through compiler augmentation," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 1–18.
- [52] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefer, M. F. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 2244–2253.
- [53] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1456–1468.
- [54] A. TehraniJamsaz, Q. I. Mahmud, L. Chen, N. K. Ahmed, and A. Jannesari, "PERFOGRAPH: A numerical aware program graph representation for performance optimization and program analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 57783–57794.
- [55] J. El Karchi, H. Chen, A. TehraniJamsaz, A. Jannesari, M. Popov, and E. Saillard, "MPI errors detection using GNN embedding and vector embedding over LLVM IR," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2024, pp. 595–607.
- [56] M. Pendleton, R. Garcia-Lebron, J.-H. Cho, and S. Xu, "A survey on systems security metrics," *ACM Comput. Surv.*, vol. 49, no. 4, pp. 1–35, 2016.
- [57] A. Chatzimparmpas, R. M. Martins, and A. Kerren, "T-viSNE: Interactive assessment and interpretation of t-SNE projections," *IEEE Trans. Vis. Comput. Graph.*, vol. 26, no. 8, pp. 2696–2714, Aug. 2020.
- [58] M. Gorbett, H. Shirazi, and I. Ray, "Sparse binary transformers for multivariate time series modeling," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 544–556.
- [59] J. Talbot, R. M. Yoo, and C. Kozyrakis, "Phoenix modular mapreduce for shared-memory systems," in *Proc. 2nd Int. Workshop MapReduce Appl.*, 2011, pp. 9–16.
- [60] X. Wei, H. Yue, S. Gao, L. Li, R. Zhang, and J. Tan, "G-SEAP: Analyzing and characterizing soft-error aware approximation in GPGUs," *Future Gener. Comput. Syst.*, vol. 109, pp. 262–274, 2020.



Bao Wen received the BE degree in software engineering, and the ME degree in electronic information from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2021 and 2024, respectively. He is currently working toward the PhD degree with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China. His current research interests include system reliability, knowledge graph and deep learning.



Jingjing Gu (Member, IEEE) received the BE degree in computer science, and the PhD degree in computer science and technology from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2005 and 2011, respectively. She is currently a professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China. From 2017 to 2018, she was a visiting scholar with the Business School, Rutgers, State University, New Jersey. She has authored or coauthored more than 60 papers in some prestigious refereed journals and conference proceedings such as the *ACM Transactions on Embedded Computing Systems*, *ACM SIGKDD*, *AAAI*, *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, and *IJCAI*. Her current research interests include system reliability, intelligent system, mobile data mining, and urban computing. She is a senior member of the CCF. She was the (senior) program committee member in many conferences.



Dazhong Shen (Member, IEEE) received the BE degree with the School of Mathematical Sciences and the PhD degree with the School of Computer Science and Technology from the University of Science and Technology of China (USTC), Hefei, China, in 2017 and 2023. He is currently an associate professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics. He has authored more than 30 journal and conference papers in the fields of Generative Model and *Natural Language Processing*, including the *ACM Transactions on Information Systems*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on Knowledge and Data Engineering*, *NeurIPS*, *ICML*, *SIGKDD*, *IJCAI*, *WWW*.



Yang Liu received the BE degree in software engineering from Jilin University, Changchun, China, in 2018, and the ME degree with the College of Computer Science and Technology from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2021. He is currently working toward the PhD degree with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China. His current research interests include system reliability, and machine learning.



Qiang Zhou received the BE and PhD degrees in software engineering from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2016 and 2022, respectively. He is currently a lecturer with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China. His current research interests include urban computing, data mining, deep learning, and spatio-temporal prediction.



Haocheng Song received the BE degree in computer science and technology from Jiangxi Normal University, Nanchang, China, in 2022. He is currently working toward the ME degree with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China. His current research interests include system reliability, and deep learning.



Fuzhen Zhuang (Senior Member, IEEE) received the BE degree from the College of Computer Science, Chongqing University, Chongqing, China, in 2006, and the PhD degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2011. He is currently a full professor with the Institute of Artificial Intelligence, Beihang University. He has authored or coauthored more than 140 papers in some prestigious refereed journals and conference proceedings such as the *Nature Communications*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Cybernetics*, *IEEE Transactions on Neural Networks and Learning Systems*, *ACM Transactions on Knowledge Discovery from Data*, *ACM Transactions on Intelligent Systems and Technology*, *Information Sciences*, *Neural Networks*, *SIGKDD*, *IJCAI*, *AAAI*, *TheWebConf*, *SIGIR*, *ICDE*, *ACM CIKM*, *ACM WSDM*, *SIAM SDM*, and *IEEE ICDM*. His research interests include transfer learning, machine learning, data mining, multitask learning, knowledge graph, and recommendation systems. He is a senior member of the CCF. He was the recipient of the Distinguished Dissertation Award of CAAI in 2013.



Xinyi Huang (Member, IEEE) received the PhD degree from the School of Computer Science and Software Engineering, University of Wollongong, Australia. He is currently a professor with the College of Computer Science and Technology/College of Software, Nanjing University of Aeronautics and Astronautics, China. He has published more than 160 research papers in refereed international conferences and journals, such as *ACM CCS*, *CRYPTO*, *ASIACRYPT*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, and *IEEE Transactions on Information Security and Forensics*. His work has been cited more than 17000 times at Google Scholar (H-Index: 68). His research interests include cryptography and information security. He is on the editorial board of the *International Journal of Information Security and Science China Information Sciences*. He has served as the program/general chair or a program committee member for more than 120 international conferences.